SQA Workflow — Design to Sign-off

Oct 10, 2026 · @Spriha

QA is involved from the moment a design is approved until the final sign-off note, so defects are prevented early rather than found late. This document describes the standard SQA workflow in an Agile SDLC, how each stage runs with and without AI, and how to handle exception cases.

Purpose, scope and roles

Scope: every feature from approved design and acceptance criteria (AC) to QA sign-off. Tool names in this document are examples only; any equivalent tool works.

Role	Responsibility
Project Manager (PM)	Tracks project timelines and sprint progress; creates tickets ahead of sprint planning
Product Owner (PO)	Owns the ticket description and AC; approves design; decides scope and priority
Designer	Owns the UI design; answers design questions
Business Analyst (BA)	Clarifies business rules and edge cases
Developer	Builds and deploys; shares the environment URL and test data; updates ticket status
QA Engineer	Reviews design and AC; writes and runs tests; logs bugs and retests; verifies deployments; writes the sign-off note
QA Lead	Reviews test coverage; approves the sign-off note

Typical tools (examples): a design tool (e.g. Figma), an issue tracker (e.g. JIRA), a test management tool (e.g. Zephyr), an API client (e.g. Postman), a CI/CD tool (e.g. Jenkins) and a wiki (e.g. Confluence).

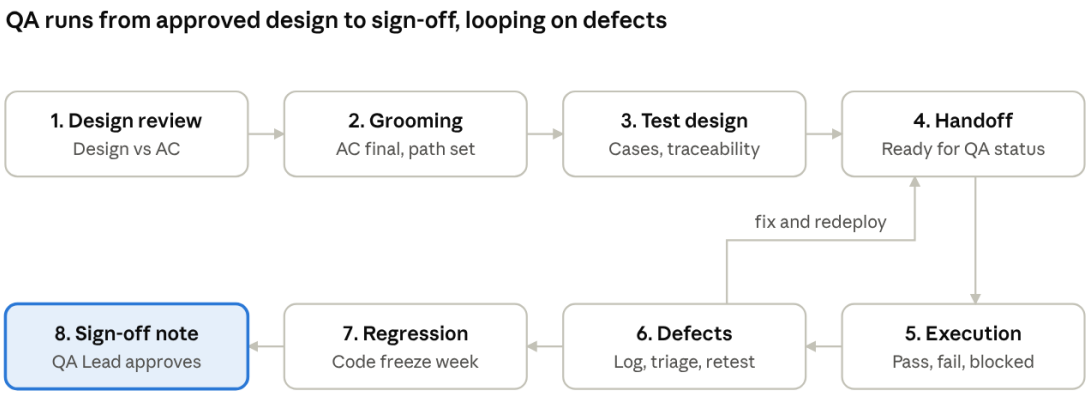
Sprint cadence

The team works in Agile sprints: tickets are picked according to capacity, and code freezes one week before release so that week is spent on regression.

Event	When	QA's part
Ticket creation	Before sprint planning	PM creates tickets in advance so sprint time is not lost to setup
Backlog grooming	Before the sprint	Stages 1 and 2
Sprint planning	Start of sprint	Tickets picked by team capacity,

Event	When	QA's part
		including QA effort
Daily stand-up	Every day	Testing progress, blockers, open bugs
Mid-sprint check	Middle of sprint	Team and PM confirm all tickets can finish so the burndown chart stays on track; QA flags testing at risk
Code freeze and regression	Final week before release	Stage 7 on the release branch
Review and retrospective	End of sprint	Show tested work; raise process issues
Release	After sign-off	Stage 8 sign-off note required

Workflow at a glance



SQA workflow · 8 stages, 1 defect loop

Stages 1 to 3 happen before and during development; a failed retest loops back through handoff until the bug is fixed, and only then does the work move to regression and sign-off.

Workflow stages

Stage	What QA does	Done when
1. Design and AC review	Compares the approved design to the AC; lists missing states (empty, error, loading, permissions) and vague AC; prepares questions	Questions attached to the ticket
2. Backlog grooming	Raises questions; assesses risk and impact on	AC final;

Stage	What QA does	Done when
	existing flows; estimates QA effort; agrees the test environment (default QA environment, or Dev for high-risk changes)	estimate and environment agreed
3. Test design	Writes test cases (positive, negative, boundary, role-based) and API tests; links test cases to the ticket and maps them to AC; gets a peer review	Reviewed test cases linked to the ticket
4. Handoff	Developer deploys, comments the URL and test data, and sets the ticket to Ready for QA; QA verifies the correct branch is deployed (deploying it through CI/CD if not) and runs a smoke check	Build accepted by QA
5. Execution	Creates a test cycle; runs UI tests manually and API tests in the API client; checks audit trails and data accuracy; records Pass, Fail or Blocked with evidence	Every test case has a result; coverage is known
6. Bugs and retest	Logs each bug linked to its failed test case; sets severity; triages with Dev and PO; retests the fix and related cases	No open critical or high bugs; the rest deferred by the PO
7. Regression	During code freeze, runs a regression cycle on the release branch; only PO-approved fixes are merged; supports the PO's acceptance check	Regression passed; PO has accepted
8. Sign-off	Compiles results and writes the sign-off note	QA Lead approves; note linked to the ticket

Working rules

- A missing AC or new scope found during development or testing becomes a separate ticket, linked to the original and with its own test cases. Tickets in progress, in QA or Done are never changed.
- A missing AC found during testing is raised as a new ticket, not as a bug.
- Every test case is linked to its ticket, and every bug is linked to its failed test case.
- UI testing is manual; API testing is done in an API client.

QA sign-off note

QA issues the note once every exit criterion is met; the QA Lead approves it and the PO receives it as the record that the work is ready for release.

- ☐ All planned test cases executed, and every AC traced to at least one passed test case
- ☐ No open critical or high bugs; any others deferred by the PO and listed in the note
- ☐ Regression passed on the release branch
- ☐ PO acceptance received

Section	Contents
Feature and release	Ticket ID, feature name, sprint, release
Environment	Environment, build or branch, test dates
Scope	What was tested (modules, roles, browsers) and what was not, with reasons
Results	Test cases total, passed, failed, blocked; API test results
Bugs	Raised, closed and deferred, with IDs of deferred ones
Recommendation	Go, or Go with known issues (each listed with its risk)
Approvals	QA Engineer, QA Lead and PO, with dates

With and without AI

The stages, owners and exit criteria are the same in both processes; AI drafts and analyses, while QA makes every decision and every test verdict.

Stage	Without AI	With AI
1. Design and AC review	QA reads the ticket and design screen by screen	AI summarises the feature and flags AC with no matching screen or vague wording; QA confirms
2. Grooming	QA takes notes and estimates from experience	AI drafts meeting notes, revised AC and likely impacted areas; the team decides
3. Test design	QA writes every test case and API request by hand	AI drafts test cases, edge cases and API requests; QA reviews each one
4. Handoff	QA checks the handoff comment and deployment	Same; AI can flag a missing URL or test data
5. Execution	QA executes and records results	Execution stays manual; AI summarises progress and helps read failed API responses
6. Bugs and retest	QA writes bug reports and searches for	AI drafts bug reports and finds likely duplicates; QA checks every field

Stage	Without AI	With AI
	duplicates	
7. Regression	QA picks regression cases from experience	AI ranks regression cases by the areas changed; QA finalises the list
8. Sign-off	QA compiles the numbers and writes the note	AI drafts the note from tool data; QA verifies every number and owns the recommendation

Guardrails when using AI

1. Never put real customer data, credentials or production data into an AI tool; use masked or synthetic data.
2. Use only company-approved AI tools.
3. QA reviews every AI draft before it is saved or shared.
4. AI never marks a test as passed; only a human execution result counts.
5. Audit trail and data-accuracy checks, and every number in the sign-off note, are verified by QA directly.

Exception scenarios

When the normal flow cannot be followed, the steps below apply instead; every scenario still ends with a sign-off note.

Scenario	How it is handled	QA's part
Urgent client requirement with no time for grooming	PO confirms the urgency; a short huddle (PO, Dev, QA) replaces full grooming; PM removes work of a similar size so the sprint stays balanced	Agree minimum AC in the huddle; write test cases while development runs; list untested areas as risks in the sign-off note
Issue reported by a client in production	A ticket is raised with severity; the fix goes on a hotfix branch and, if critical, is released outside the normal cycle	Reproduce the issue first; test the fix plus a targeted regression of the affected area; add a test case so future regression covers it; issue a short hotfix sign-off
Critical bug found during code freeze	PO decides whether to fix it now or defer it to the next release	If fixed: retest and rerun affected regression. If deferred: list it as a known issue
Ticket cannot finish within the sprint	Flagged at stand-up or the mid-sprint check; PM and PO split it or carry it over	Sign off only the tested part; untested scope moves with the carried-over ticket
Wrong branch	QA redeploys the correct branch	Mark affected test cases Blocked and

Scenario	How it is handled	QA's part
deployed or environment down	through CI/CD; an environment outage is escalated to Dev or DevOps	flag them at stand-up
Incomplete handoff (no URL, test data or AC)	Ticket is returned to the developer or PO	Testing starts only once the gap is fixed
Release must go out with open bugs	PO accepts the risk in writing	Recommend Go with known issues, listing each open bug and any workaround